



Patron de comportamiento: VISITOR

*Sin Visitor, agregar nuevas operaciones puede hacer
que el código crezca y requiere más clases*

Sofia Tobon Atehortua, Veronica Loaiza Gomez | Ingenieria de Sistemas

¿Que es Visitor?

Visitor es un patrón de diseño de comportamiento que permite realizar diferentes operaciones sobre un grupo de objetos sin modificar las clases de esos objetos.

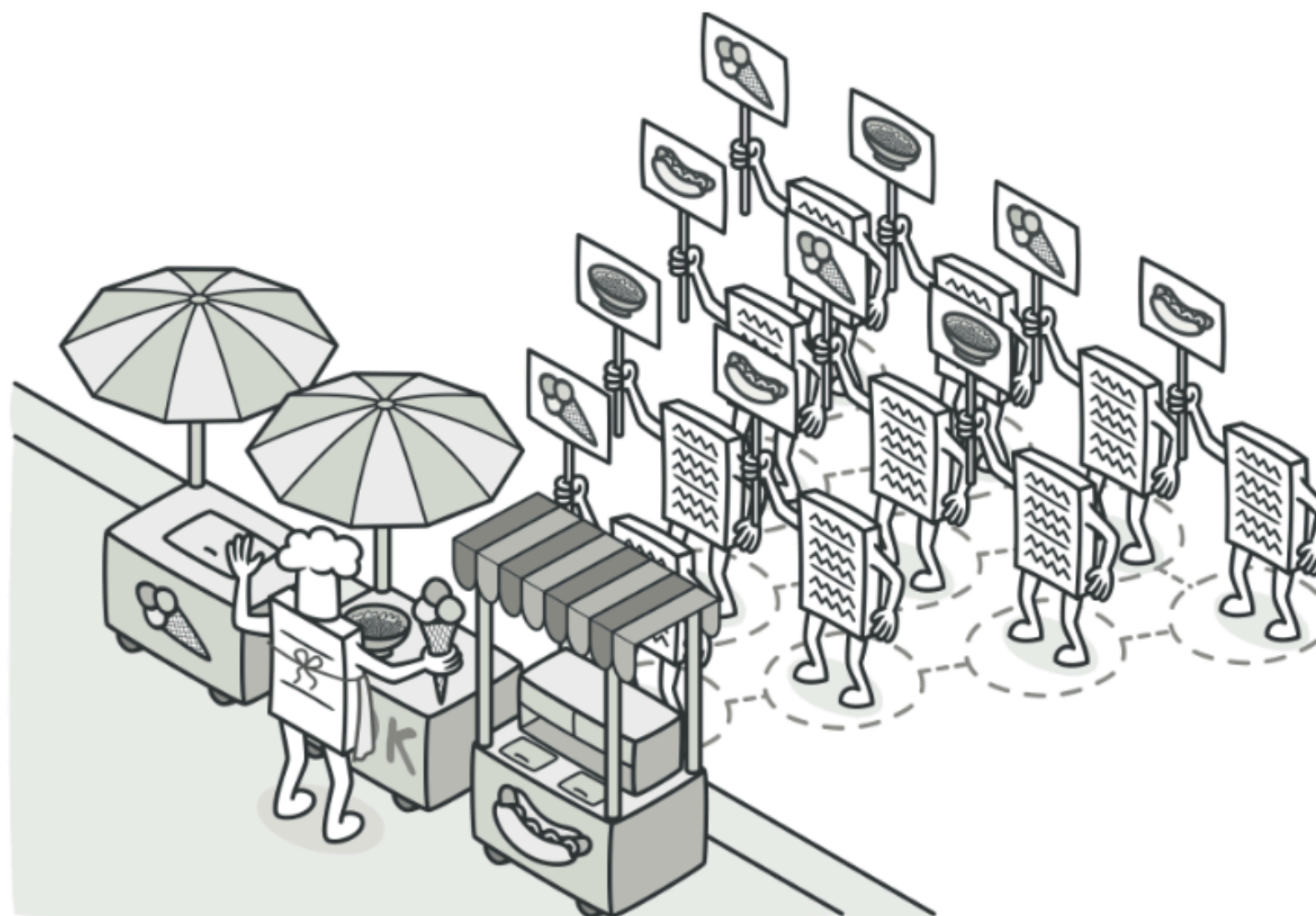
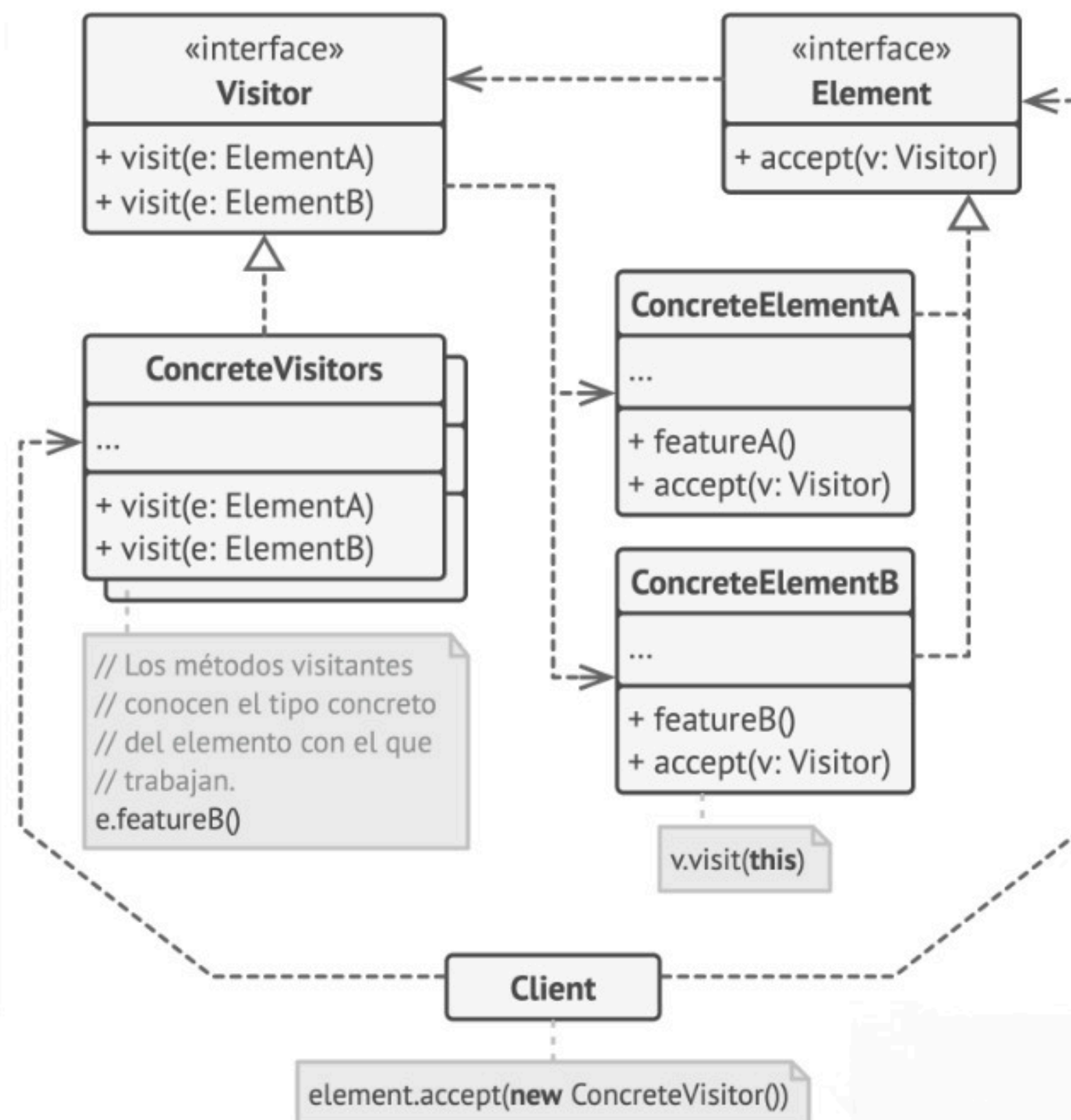


Diagrama de clase de visitor



... por la fe y la ciencia

Concepto clave de visitor: Doble despacho

El doble despacho permite que el programa determine qué operación realizar dependiendo de:

- ***El tipo de producto*** que recibe al visitante.
- ***El tipo de operación*** que realiza el Visitor.

A la Verdad, por la fe y la ciencia

Aplicabilidad

- Utiliza el patrón Visitor cuando necesites realizar una operación sobre todos los elementos de una compleja estructura de objetos (por ejemplo, un árbol de objetos).
- Utiliza el patrón Visitor para limpiar la lógica de negocio de comportamientos auxiliares.
- Utiliza el patrón cuando un comportamiento solo tenga sentido en algunas clases de una jerarquía de clases, pero no en otras.

¿Como implementar visitor?

Crear las interfaces

IVisitor: define los métodos Visitar() para cada tipo de elemento.

IProducto: define el método Aceptar().

Implementar los elementos

Ropa y Electronico implementan IProducto.

Cada uno redirige la llamada al Visitor correspondiente.

Crear el Visitor concreto

CalculadorImpuesto implementa IVisitor.

Define qué operación realizar para cada producto.

Usar el Visitor

El cliente crea los productos y el Visitor.

Luego, cada producto recibe el Visitor mediante Aceptar().

Producto → Aceptar(Visitor) → Visitar(Producto)

Al la Verdad. por la fe y la ciencia

Visitor con otros patrones

- Visitor como una versión potente del patrón Command. Sus objetos pueden ejecutar operaciones sobre varios objetos de distintas clases.
- Puedes utilizar el patrón Visitor para ejecutar una operación sobre un árbol Composite entero.
- Puedes utilizar Visitor junto con Iterator para recorrer una estructura de datos compleja y ejecutar alguna operación sobre sus elementos, incluso aunque todos tengan clases distintas.

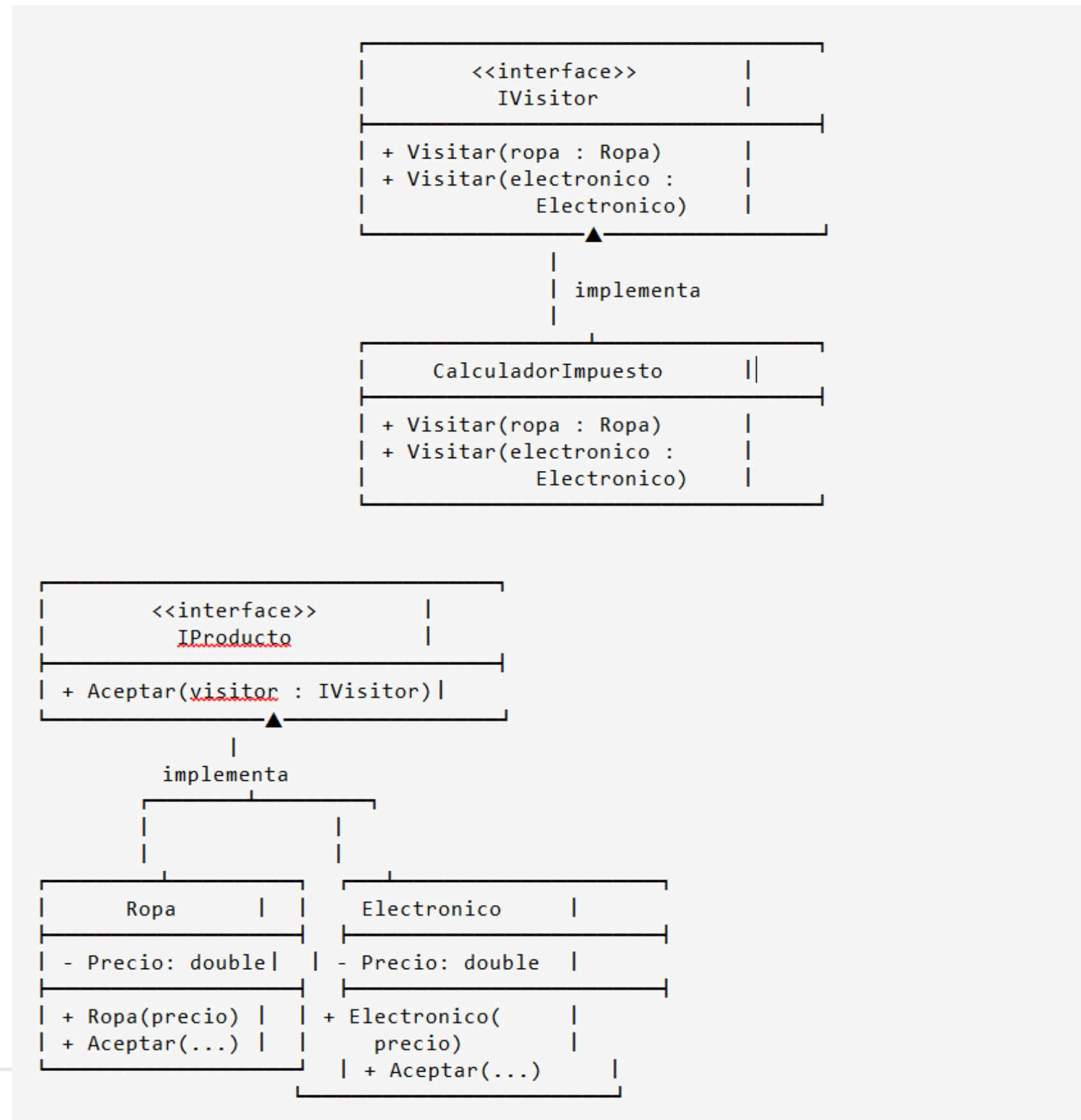
Ventajas

- Principio de abierto/cerrado. Puedes introducir un nuevo comportamiento que puede funcionar con objetos de clases diferentes sin cambiar esas clases.
- Principio de responsabilidad única. Puedes tomar varias versiones del mismo comportamiento y ponerlas en la misma clase.
- Un objeto visitante puede acumular cierta información útil mientras trabaja con varios objetos. Esto puede resultar útil cuando quieras atravesar una compleja estructura de objetos, como un árbol de objetos, y aplicar el visitante a cada objeto de esa estructura.

Desventajas

- Debes actualizar todos los visitantes cada vez que una clase se añada o elimine de la jerarquía de elementos.
- Los visitantes pueden carecer del acceso necesario a los campos y métodos privados de los elementos con los que se supone que deben trabajar.

Diagrama de clase del código



por la fe y la ciencia

Referencias bibliograficas

Visitor. (s. f.). <https://refactoring.guru/es/design-patterns/visitor>





¡Gracias!